React js
Lecture 2

By Mona Soliman

Agenda

- Integrating with Ui libraries
- Icons libraries
- React Hooks (useState , useEffect , useRef)
- Conditional Rendering & rendering multiple components
- Difference between State and Props

Ui libraries

- Bootstrap
- React Bootstrap
- Tailwind Css

  https://ui.shadcn.com/

  https://www.radix-ui.com/

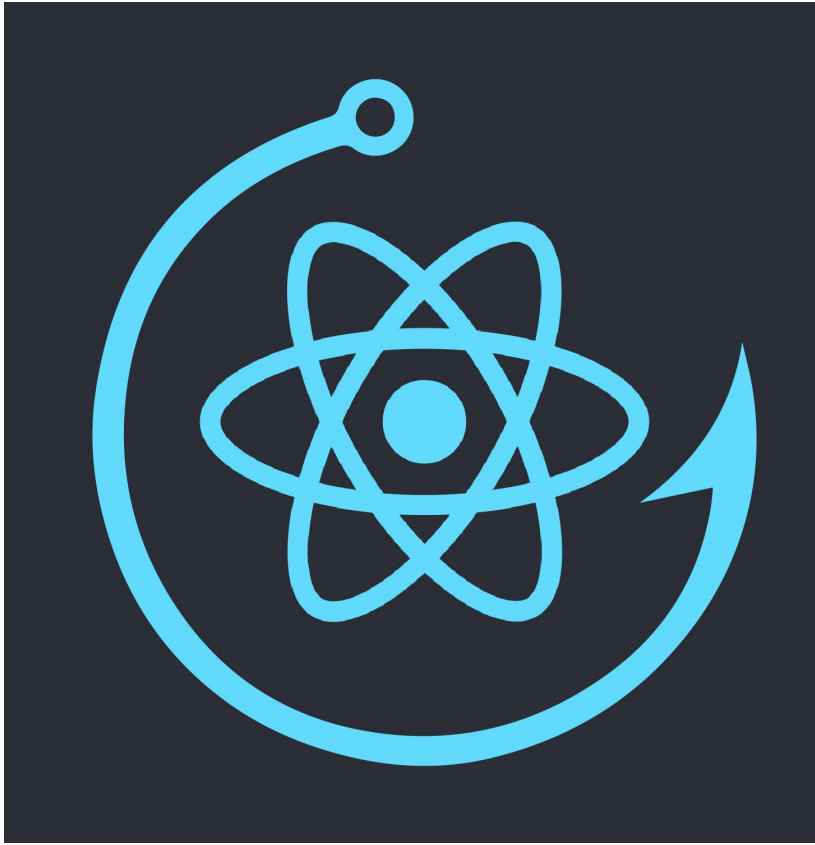
- Material Ui
- Chakra Ui
- Ant design

Icons libraries

- React Icons
- Material Ui Icons
- Bootstrap Icons
- Feather Icons
- Font Awesome Icons
- Hero Icons

React Hooks:

React Hooks are functions introduced in React 16.8 that allow you to use state and other React features without writing a class. They enable you to hook into React's state and lifecycle features from functional components, making it easier to manage state and side effects.



React Hooks:

1- useState

useState is a hook that lets you add React state to functional components.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

React Hooks:

2- useEffect

useEffect is a hook that lets you perform side effects in functional components. It's similar to componentDidMount, componentDidUpdate, and componentWillUnmount in class components.

```jsx
import React, { useEffect, useState } from 'react';

function Example() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    document.title = `You clicked ${count} times`;
  }, [count]); // Only re-run the effect if count changes

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={() => setCount(count + 1)}>
        Click me
      </button>
    </div>
  );
}
```

React Hooks:

3- useRef

useRef is a React Hook that lets you reference a value that’s not needed for rendering.

the useRef hook is primarily used for accessing and interacting with DOM elements directly, without causing a re-render when the reference value changes. It's also used to persist values across renders without triggering a re-render.

React Hooks:

1. Accessing DOM Elements

```jsx
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null);

  const handleFocus = () => {
    inputRef.current.focus(); // Focus the input element
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Focus on me!" />
      <button onClick={handleFocus}>Focus Input</button>
    </div>
  );
}
```

React Hooks:

2. Persisting Values Across Renders

```jsx
import React, { useRef, useState, useEffect } from 'react';

function Timer() {
  const [count, setCount] = useState(0);
  const countRef = useRef(count);

  useEffect(() => {
    countRef.current = count;
  }, [count]);

  useEffect(() => {
    const timer = setInterval(() => {
      setCount((prevCount) => prevCount + 1);
      console.log('Current count:', countRef.current); // Logs the latest count value
    }, 1000);

    return () => clearInterval(timer);
  }, []);

  return <div>Count: {count}</div>;
}

export default Timer;
```

Rendering Multiple Elements

To render multiple elements in React, you can use the Array's map functionality. Simply map all your objects into React fragments, so that your Function component can make use of it. But don't forget to set a unique key prop!

```jsx
import React from 'react';

function App() {
  const items = ['Item 1', 'Item 2', 'Item 3', 'Item 4'];

  return (
    <div>
      <h1>Item List</h1>
      <ul>
        {items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    </div>
  );
}

export default App;
```

Rendering Multiple Elements

What is the **KEY Prop** ?

You need to give each array item a `key` — a string or a number that uniquely identifies it among other items in that array.

- **Keys must be unique among siblings.** However, it's okay to use the same keys for JSX nodes in *different* arrays.
- **Keys must not change** or that defeats their purpose! Don't generate them while rendering.

https://react.dev/learn/rendering-lists#keeping-list-items-in-order-with-key

Conditional rendering

Conditional rendering in React works the same way conditions work in JavaScript. Use JavaScript operators like if or the conditional operator to create elements representing the current state, and let React update the UI to match them.

You can use ternary operator or && to implement conditional rendering

Class component vs Functional component

	Function Components	Class Components
Definition	JavaScript function that returns a JSX element	JavaScript class that extends React.Component and has a render() method that returns a JSX element
State	Uses React Hooks to manage state	Has built-in state management using this.state
Lifecycle Methods	Uses React Hooks to handle lifecycle events like `useState`, `useEffect`, `useMemo`.	Uses class lifecycle methods like `componentDidMount`, `componentDidUpdate`, and `componentWillUnmount`
Props	Props are passed as a parameter to the function	Props are accessed through this.props
Render	The component is defined within the function body and returned as a JSX element	The render() method is required and returns a JSX element
Code Clarity	Simple and concise	More verbose and harder to read
Performance	Sets you up to follow best practices	Easier to cause performance issues

Thank you